

ACCELERATED BUILD PLAYBOOK

7

MeetAI in 7 Days

Seven high-level, copy-ready prompts to move an existing repository from foundation to a tested, deployed, portfolio-ready meeting intelligence platform.

DAY 1	DAY 2	DAY 3	DAY 4	DAY 5	DAY 6	DAY 7
-------	-------	-------	-------	-------	-------	-------

Designed from the complete instruction.md and all 9 pages of the MeetAI Detailed Project, Architecture & Implementation Guide.

Version 1.0 | 08 September 2026

How to use this playbook

The source plan recommends 15 days at 4-6 focused hours per day. This seven-day version is an aggressive compression. Correctness, evidence and a working core outrank the calendar.

Operating contract

- Run one prompt per day, in order. Each prompt is standalone but assumes the previous day's acceptance gate passed.
- Start from the repository root. Give the agent terminal access, the current codebase and the previous day's handoff report.
- Require inspection and a file-level plan before edits. Preserve working code, unrelated changes and user data.
- Do not advance past a failed dependency. Mark the day PARTIAL, carry the unfinished core forward and cut polish before cutting correctness.
- Make small, coherent commits only after relevant tests and runtime checks pass. Never force-push or erase uncommitted work.
- When Docker, an API key, certificates or a cloud target are unavailable, record the exact blocker. Do not substitute an unverified claim.

Rules repeated across all seven prompts

Architecture	Routes parse HTTP and inject dependencies; services hold business logic; repositories own SQL; schemas use Pydantic; models use SQLAlchemy; WebSocket and worker logic stay in their dedicated layers.
Quality	Python is fully typed; TypeScript stays strict with no casual any; multi-table writes are atomic; every feature includes positive and negative tests.
Safety	No secrets in Git, no raw stack traces, no AI work in request handlers, no destructive migrations, no skipped tests and no fabricated metrics or success claims.
Scope	No WebRTC, calendar sync, enterprise RBAC/SSO, diarization, live AI assistant, vector/RAG search or analytics dashboard. Build uploaded-audio, OWNER/PARTICIPANT, timestamped transcript and PostgreSQL search flows.

Seven-day dependency map

The compression groups adjacent source phases into complete vertical slices. Security and testing happen every day, then receive dedicated consolidation before release.

DAY	FOCUS	SOURCE DAYS	NON-NEGOTIABLE GATE
1	Foundation + data layer	1-2	Health, UI, Postgres/Redis and migration verified
2	Authentication + meetings	3-4	Two-user auth and authorization flows pass
3	Real-time collaboration	5-7	Persistent chat, presence and cross-worker delivery pass
4	Audio + AI pipeline	8-10	Upload reaches stored transcript and structured AI output
5	History, search + security	11-12	Authorized search and abuse controls pass
6	Tests + production deployment	13-14	Full test matrix and six-service stack pass
7	Release acceptance + portfolio	15 + closure	Deployed E2E evidence and honest metrics complete

Critical path

Foundation -> data model -> authentication -> meeting authorization -> WebSocket rooms -> audio job -> transcript -> summary -> search -> full tests -> deployment -> portfolio evidence

Daily handoff rule

Every prompt ends with the same evidence package: status, changed files, architecture/data flow, exact commands and results, migration impact, security notes, residual risks, and the next day's prerequisites. A plain 'done' is not an acceptable handoff.

Decisions to lock before implementation

The two source documents are aligned on architecture but leave a few implementation choices open. Day 1 should record these decisions once so later prompts do not create incompatible contracts.

TOPIC	LOCKED DECISION	WHY IT MATTERS
API paths	Choose one canonical versioned REST prefix and preserve any compatibility aliases deliberately.	The sources alternate among /api, /api/v1 and unversioned examples.
Database URL	Use localhost for host-run tools and Docker service DNS (db) for container traffic; keep one database name.	A host URL cannot connect from another container.
DB drivers	Use asyncpg for the async application engine; keep the sync driver only for tools that require it.	Mixed implicit engine modes create migration and worker surprises.
Completion	Define COMPLETED as the entire required transcript plus intelligence transaction being durable, or add explicit stage states.	The source roadmap can otherwise mark transcription complete too early.
Test evidence	All selected tests must pass; report coverage as a separate measured percentage.	An 80 percent pass rate is not a release-quality test gate.
WebSocket join	Specify an authenticated join acknowledgement while retaining the documented event envelope and close code 4001.	The event table defines a request but no positive acknowledgement.
Upload/provider	Make format allowlist, size, storage, retention and provider model names environment-configurable and tested.	Those details vary by environment and provider lifecycle.
Profile scope	Include minimal profile read/update because profile management is a stated success criterion.	The short roadmap mentions current-user retrieval but not update.

Record these decisions in code, tests and README. Do not let different services or daily agents silently choose different contracts.

DAY 1 / SOURCE ROADMAP DAYS 1-2

Foundation and complete data layer

One standalone prompt. Paste it into the coding agent from the repository root.

Turn the existing repository into a verified React/FastAPI/PostgreSQL/Redis foundation with the complete durable schema. Do not assume Phase 0 is finished, and do not silently use SQLite as proof of PostgreSQL readiness.

COPY-READY DAILY PROMPT	
ROLE	You are the lead full-stack engineer inside the existing MeetAI repository. Complete Day 1 autonomously, but preserve working code and all unrelated user changes.
INSPECT FIRST	Read repository instructions, inspect Git status, existing backend/frontend/config/migrations/tests, and run the current checks. Report the plan and exact files before editing. Reuse correct work instead of rescaffolding.
FOUNDATION	Verify FastAPI with title MeetAI API, version 1.0.0, restricted CORS, /health, /api/health and docs. Verify React + Vite + strict TypeScript + Tailwind + Router. Provide environment validation without exposing secrets. Compose must use PostgreSQL 16, Redis 7, persistent volumes and the single database name meetai_db.
DATA LAYER	Implement SQLAlchemy 2 models for users, refresh_tokens, meetings, meeting_participants, messages, transcripts, transcript_segments, meeting_summaries and action_items. Add typed enums, timestamps, foreign keys, uniqueness, indexes, cascades and relationships. Create matching Pydantic v2 schemas plus repository/service foundations without putting SQL in routes.
MIGRATIONS	Wire Alembic metadata, create the initial meaningful revision, review generated SQL, and verify upgrade, downgrade and re-upgrade against PostgreSQL without deleting existing user data. Multi-model operations must be transaction-ready.
TEST + VERIFY	Install dependencies in project-local environments. Run backend tests, import/compile checks, frontend production build, Compose validation, service health, migration checks and real HTTP requests. If Docker's Linux engine is unavailable, record the blocker and mark infrastructure PARTIAL; do not substitute SQLite.
ACCEPTANCE	PASS only when health responses and docs load, Vite builds and serves, PostgreSQL and Redis are reachable, migration round-trip works, all nine tables have correct constraints, tests pass and no secret is tracked.
HANDOFF	Return PASS/PARTIAL/FAIL, changed files, schema relationship summary, migration revision, exact commands/results, unresolved risks and Day 2 prerequisites. Commit only the verified Day 1 slice.

DAY 2 / SOURCE ROADMAP DAYS 3-4

Authentication and meeting lifecycle

One standalone prompt. Paste it into the coding agent from the repository root.

Deliver the first complete product slice: two users can authenticate, manage secure sessions, create or join meetings, and see authorized meeting data in the React application.

COPY-READY DAILY PROMPT	
ROLE	Continue as the MeetAI lead engineer. Start from the verified Day 1 handoff. Do not redesign the data layer unless a demonstrated defect requires a migration.
INSPECT FIRST	Review current models, schemas, repositories, services, routes, frontend auth state and tests. Trace the intended register -> login -> dashboard -> meeting flow, then present a scoped plan and file list.
AUTH	Implement registration, strong password hashing, login, short-lived JWT access tokens, cryptographically random refresh tokens stored only as hashes, refresh rotation, revocation on logout, current-user/profile endpoints and inactive-user checks. Prevent token replay and never log credentials or tokens.
MEETINGS	Implement create, list, detail, join, leave, cancel and archive operations with valid status transitions. Create the owner and OWNER participant atomically. Enforce OWNER/PARTICIPANT permissions on every resource; return clean 401/403/404 errors without leaking database details.
FRONTEND	Build accessible Login, Register, protected routes, session restoration/refresh, logout, Dashboard lists/status badges, meeting creation/join flows and room navigation. Keep API types aligned with Pydantic responses and handle loading, empty and error states.
TEST + VERIFY	Add service, repository, API and UI tests for happy paths and duplicate email, bad password, expired/invalid access, refresh rotation/reuse, revoked refresh, unauthorized access, invalid meeting transition and owner/participant boundaries. Run migrations, full backend tests, frontend tests/build and real API smoke flows with two users.
ACCEPTANCE	PASS only when two users can register/login; refresh and logout behave securely; unauthorized calls return 401; forbidden meeting access returns 403; meeting lifecycle state is durable; the dashboard reflects correct accessible meetings; all tests pass.
HANDOFF	Report status, changed files, access/refresh token data flow, meeting transaction and authorization rules, commands/results, remaining risks and the exact authenticated endpoints Day 3 will use. Commit only after verification.

DAY 3 / SOURCE ROADMAP DAYS 5-7

WebSockets, chat, presence and Redis reliability

One standalone prompt. Paste it into the coding agent from the repository root.

Create a reliable two-user meeting room with authenticated structured events, durable chat, live presence, Redis cross-worker delivery and reconnect behavior - without WebRTC.

COPY-READY DAILY PROMPT	
ROLE	Implement the real-time collaboration vertical slice on top of the verified auth and meeting authorization layers. Preserve the REST contracts and database history.
INSPECT FIRST	Trace WebSocket upgrade, token validation, meeting membership, room state, message persistence, Redis and frontend reconnect paths. Propose the event schemas and files before changing code.
PROTOCOL	Use one typed envelope: {event, data, timestamp}. Support connection.join, presence.joined, presence.left, chat.message, typing.start, typing.stop, meeting.state and controlled error. Authenticate at handshake or first join, authorize membership and close unauthorized sockets with code 4001.
SERVER	Build room connection management, clean join/leave/disconnect handling, heartbeat ping/pong and stale cleanup. Persist chat to PostgreSQL before broadcast. Add bounded history retrieval. Integrate Redis Pub/Sub for multiple API workers, prevent self-echo/duplicate rebroadcast and clean subscriptions when rooms empty.
CLIENT	Implement a typed WebSocket hook with exponential-backoff reconnect, token refresh coordination, resubscription/state recovery and deliberate retry limits. Build meeting room chat, history, participant presence, typing indicator, lifecycle state, accessibility and clear offline/error UI.
TEST + VERIFY	Test valid/expired/unauthorized sockets, every event schema, persistence-before-broadcast, two clients, disconnect cleanup, heartbeat timeout, reconnect and Redis cross-worker delivery. Verify a page refresh reloads chat and a backend restart triggers recovery without duplicate messages.
ACCEPTANCE	PASS only when two authorized sessions exchange ordered persistent messages, see accurate presence/typing/state, unauthorized users are rejected, multiple workers deliver once, reconnect recovers, all tests pass and no orphan room state remains.
HANDOFF	Report status, event catalog, connection/message/Redis flow, exact multi-client and multi-worker evidence, changed files, commands/results, known scaling limits and Day 4 integration points. Commit the verified slice.

DAY 4 / SOURCE ROADMAP DAYS 8-10

Audio, Celery, transcription and AI intelligence

One standalone prompt. Paste it into the coding agent from the repository root.

Complete the asynchronous intelligence pipeline from an authorized audio upload to stored transcript segments, summary, decisions and action items rendered in the meeting UI.

COPY-READY DAILY PROMPT

ROLE	Implement the audio/AI pipeline as a durable background workflow. No transcription, transcoding or language-model call may execute inside a FastAPI request handler.
INSPECT FIRST	Review transcript/summary models, storage settings, Celery configuration, Redis, OpenAI SDK usage, meeting permissions and frontend meeting view. Define the state machine and transaction boundaries before edits.
UPLOAD	Add an authenticated meeting audio endpoint. Enforce participant authorization, configured size limit (25 MB unless configuration says otherwise), extension plus actual MIME/content checks for MP3/WAV and intentionally supported M4A, safe generated filenames, path traversal prevention and controlled errors. Persist UPLOADED before enqueueing.
WORKER	Configure Celery/Redis and an idempotent task that atomically claims work, sets PROCESSING, calls the configured OpenAI transcription model with explicit timeout, and stores raw text plus ordered timestamp segments. Retry only transient failures with bounded exponential backoff; terminal failures set FAILED with sanitized diagnostics; clean temporary files.
INTELLIGENCE	After transcription, request validated structured output for overview, key decisions and action items. Validate with Pydantic before atomic persistence. Re-delivery must replace or safely reuse results without duplicate segments, summaries or items. Set COMPLETED only after all required records commit.
FRONTEND	Add upload/progress UI, polling or WebSocket state updates, retry guidance, timestamped transcript, summary, decisions and action-item views. Keep failed processing understandable without showing provider secrets or internal traces.
TEST + VERIFY	Mock provider calls in automated tests. Cover invalid/oversize/unauthorized files, enqueue-after-commit, every state transition, transient retry, terminal failure, idempotent redelivery, atomic rollback and cleanup. Run a real-key smoke test only when a valid key is configured; otherwise label it unverified.
ACCEPTANCE	PASS only when an allowed fixture follows UPLOADED -> PROCESSING -> COMPLETED and the UI renders stored transcript and structured output; rejection and FAILED/retry paths are proven; duplicate delivery is harmless; API remains responsive; all tests pass.
HANDOFF	Report status, upload/job/provider/database/UI flow, state transition evidence, changed files, commands/results, mocked versus live verification, cost/privacy risks and Day 5 query contracts. Commit only verified work.

DAY 5 / SOURCE ROADMAP DAYS 11-12

History, PostgreSQL search and security hardening

One standalone prompt. Paste it into the coding agent from the repository root.

Make completed meeting intelligence discoverable only to authorized users, then harden every REST, WebSocket, file and background-job boundary before the full test and deployment day.

COPY-READY DAILY PROMPT

ROLE	Deliver authorized history/search and a repository-wide security/reliability hardening pass. Fix demonstrated issues with the smallest coherent changes and regression tests.
INSPECT FIRST	Map access control from user -> participant -> meeting -> message/transcript/summary/action item. Audit API, WebSocket, storage, worker, logs, settings, dependency versions and existing tests. Rank findings Critical/High/Medium and plan before editing.
HISTORY + SEARCH	Implement paginated meeting history and transcript inspection for accessible meetings only. Add PostgreSQL text search over meeting titles and transcript text/segments with appropriate indexes, bounded query/limit/offset or cursor rules, date/status filters and safe snippets/highlighting. Do not introduce embeddings, vector stores or RAG.
FRONTEND	Build History and search views with filters, result context, transcript navigation, pagination, URL state, keyboard/accessibility support and loading, empty, error and forbidden states. Keep API types strict.
HARDEN	Verify password/token hashing, expiry/rotation/revocation, resource checks everywhere, explicit production CORS, rate limits on register/login/upload/WebSocket, Pydantic input/output validation, file content/size/path safety, safe client errors/log redaction, secrets only in environment, atomic writes, provider timeouts/retries, job idempotency and pagination caps.
TEST + VERIFY	Add authorization, injection-like input, pagination bounds, rate limit, CORS, upload abuse, path traversal, safe-error, retry and duplicate-job regression tests. Prove a spoken keyword returns only meetings visible to the caller and a nonparticipant receives 403. Run dependency and secret-hygiene checks without printing secrets.
ACCEPTANCE	PASS only when history/search are correct, indexed, paginated and authorization-scoped; abuse controls work; errors reveal no internals; secrets are untracked; all new findings at Critical/High are resolved or explicitly block release; all tests/builds pass.
HANDOFF	Report status, search query/index strategy, authorization matrix, findings and fixes, changed files, commands/results, unresolved Medium risks and the full Day 6 test/deployment prerequisites. Commit verified changes.

DAY 6 / SOURCE ROADMAP DAYS 13-14

Comprehensive testing and production deployment

One standalone prompt. Paste it into the coding agent from the repository root.

Convert daily confidence into release evidence, then package the six-service system behind Nginx with production-safe configuration and a repeatable deployment smoke test.

COPY-READY DAILY PROMPT	
ROLE	Act as test lead and deployment engineer. Do not weaken assertions, skip failures or call a container configuration deployed until it has actually run in the available environment.
INSPECT FIRST	Inventory the test matrix, coverage gaps, migration history, Dockerfiles, Compose, Nginx, health checks, environment contract and deployment target. Prioritize critical-path and failure-mode gaps, then list files and commands.
TEST MATRIX	Complete unit tests for services/security/schemas; API tests for auth, meetings, audio and search including negative authorization; database tests for constraints/cascades/transactions and migration upgrade/rollback; WebSocket auth/events/disconnect/reconnect/cross-worker tests; Celery enqueue/retry/FAILED/idempotency tests with provider mocks; frontend component/integration tests; and the critical two-user E2E flow.
EVIDENCE	Run every suite from a clean environment. All tests must pass. Measure and report actual coverage for core modules; target meaningful coverage around 80 percent or better when feasible, but never confuse a coverage target with test pass rate and never invent a number.
PRODUCTION	Provide production builds and six services: frontend, backend, db, redis, celery_worker and nginx. Use health checks, persistent volumes, restart behavior, least-privilege/non-root containers where practical, a clear migration/startup strategy and production environment validation. Nginx must serve the app, proxy /api, upgrade /ws and terminate HTTPS using supplied certificates/configuration.
VERIFY	Build from scratch, launch with one documented Compose command, wait for healthy status, inspect logs, run health and frontend smoke checks, verify WebSocket upgrade and execute the two-user critical flow through Nginx. Test migration behavior and service restart. If Docker/certs/cloud are unavailable, mark those checks PARTIAL rather than simulating success.
ACCEPTANCE	PASS only when all test layers pass, production images build, all six services become healthy, Nginx routes HTTP/WebSocket correctly, HTTPS works on the deployment target, migrations are repeatable, no secret is baked or committed and the E2E critical path succeeds.
HANDOFF	Report status, full command/result matrix, actual coverage, image/service health, deployment URL or precise blocker, changed files, rollback/restore notes, residual release risks and the Day 7 acceptance checklist. Commit only reproducible work.

DAY 7 / SOURCE ROADMAP DAY 15 + CLOSURE

Final acceptance, resilience and portfolio readiness

One standalone prompt. Paste it into the coding agent from the repository root.

Prove the deployed system end to end, fix release blockers, polish the user experience and produce honest portfolio evidence that can be explained in an interview.

COPY-READY DAILY PROMPT

ROLE	Act as release owner. Start from the deployed Day 6 handoff. Freeze feature scope: fix only release blockers, correctness defects, accessibility problems and documentation gaps.
ACCEPTANCE RUN	Execute the deployed two-session journey: register/login -> create meeting -> second user joins -> presence/chat/typing -> refresh history -> upload allowed audio -> observe state transitions -> view transcript/summary/decisions/action items -> search a spoken keyword. Capture commands, logs and screenshots as evidence.
RESILIENCE	Exercise unauthorized resource access, expired/revoked tokens, backend restart/reconnect, Redis/worker restart, duplicate task delivery, provider timeout and terminal AI failure -> FAILED -> controlled retry. Verify health endpoints, safe logs, data durability and clean recovery. Fix failures and add regression tests.
POLISH	Refine responsive layouts, consistent visual hierarchy, keyboard navigation, focus states, contrast, loading/empty/error states and intentional dark/light behavior if already supported. Remove dead code and placeholders without destabilizing proven flows.
DOCUMENT	Complete README/setup/deployment/migration/test/troubleshooting/limitations guidance. Add architecture, ERD and REST/WebSocket/job data-flow diagrams plus representative screenshots. Document security decisions, tradeoffs and scaling path. Keep future-scope exclusions explicit.
MEASURE	Measure, never invent: representative API p95 latency, WebSocket event latency, tested concurrent participants, audio-to-COMPLETED duration, test coverage and any observed availability period. State environment, sample size and method; label unavailable metrics honestly.
INTERVIEW	Produce a concise explanation of JWT/refresh rotation, meeting permissions, WebSocket room lifecycle, Redis roles, Celery/idempotency, transcript/summary schema, PostgreSQL search, Docker/Nginx and horizontal scaling. Ensure each statement matches implemented code.
ACCEPTANCE	Return a final Definition of Done table with PASS/PARTIAL/FAIL and direct evidence for authentication, meetings, real-time, AI, search, security, testing, deployment, portfolio and understanding. Release only with no unresolved Critical/High defect, clean tests/builds, secret hygiene and a reproducible demo.
HANDOFF	List changed files, final commit(s), exact verification commands/results, deployment/demo location, screenshots/metrics, known limitations and next safe improvements. After a clean release commit, synchronize requested branches only through fetched, ancestry-checked fast-forward updates or deliberate non-destructive merges; never force-push or overwrite divergence. Do not claim completion for unavailable external verification.

Final definition of done

Use this after Day 7. Every PASS needs observable evidence; PARTIAL must name the missing verification and owner.

AREA	DONE WHEN	EVIDENCE
Authentication	Register, login, rotate refresh, logout, profile and authorization work.	API tests + two-user smoke
Meetings	Create, join, leave, cancel, archive and history persist correctly.	DB/API/UI evidence
Real-time	Two users chat and see presence with reconnect and cross-worker delivery.	WS integration + live run
AI pipeline	Audio produces durable transcript, segments and structured intelligence.	Worker tests + fixture run
Search	Authorized, paginated PostgreSQL search returns meeting/transcript matches.	Query plan + API/UI tests
Security	Token, input, file, rate-limit, CORS, error and secret controls are proven.	Negative tests + audit
Testing	All unit/API/DB/WS/worker/frontend/E2E tests pass; actual coverage reported.	Clean test logs
Deployment	Six services run behind Nginx/HTTPS with health and restart checks.	Compose status + smoke
Portfolio	README, diagrams, screenshots and measured metrics are accurate.	Reviewed artifacts
Understanding	The owner can explain every major flow and tradeoff from the code.	Interview walkthrough

Release principle: the best portfolio version is not the one with the most code. It is the one that works, is tested, is deployed, is secure enough for its stated context, and can be explained clearly.

Daily handoff template

Require the coding agent to end every day with this compact report. It makes the next prompt state-aware and prevents hidden carryover.

COPY-READY DAILY PROMPT	
STATUS	PASS / PARTIAL / FAIL, with one-sentence reason
DELIVERED	Implemented behavior and user-visible outcome
FILES	Created and modified files; migration revision if any
DATA FLOW	Request/event/job path and transaction boundaries
VERIFICATION	Exact commands, results, endpoints and runtime evidence
SECURITY	Authorization, secret, validation and failure-mode notes
RISKS	Known limitations, unverified external dependencies and follow-up
NEXT DAY	Contracts and prerequisites the next prompt can rely on

Scope lock

- Use uploaded audio, not native WebRTC streaming.
- Use in-app meeting scheduling, not Google or Outlook calendar sync.
- Use OWNER/PARTICIPANT roles, not enterprise multi-tenant RBAC or SSO.
- Use timestamped transcript segments, not speaker diarization.
- Use post-meeting intelligence, not a live AI meeting or voice assistant.
- Use PostgreSQL text search, not a vector database, embeddings or RAG.
- Measure real behavior; do not add an analytics dashboard or invent metrics.

Source basis

This playbook synthesizes the complete repository instruction.md and all nine pages of MeetAI_Detailed_Implementation_Guide.pdf. It preserves the source architecture, success criteria, event protocol, data model, AI state machine, security checklist, test matrix, deployment target, scope exclusions and final definition of done while compressing the schedule from 15 days to seven.